

Guard Tour Capability Comparison

AegisLead Guard Tour vs. Silvertrac and Staffr / TrackTik

Prepared: September 14, 2026

Purpose and Method

This report compares the Guard Tour capability built into our platform against the two commercial platforms profiled in the *Guard Tour Software Research Report* (Silvertrac Software and Staffr / TrackTik, both owned by Trackforce).

The two source columns are taken from that research report, which was itself compiled from vendor product documentation, help-center articles, and app-store listings. Our column is different in kind: it was derived by reading our own source code directly — the Prisma schema, the NestJS controllers and services, and the Next.js pages listed below — so it reflects what the system actually does today rather than what a feature list claims.

Area	Primary source read
Data model	<code>backend/prisma/schema.prisma</code> — Checkpoint, PatrolRoute, PatrolRouteCheckpoint, PatrolRun, PatrolEvent, PatrolEvidence, EmergencyAlert
Admin API	<code>backend/src/patrols/patrols.controller.ts</code> , <code>patrols.service.ts</code>
Guard API	<code>backend/src/patrols/guard-patrols.controller.ts</code>
Client API	<code>backend/src/patrols/client-patrols.controller.ts</code>
Offline sync	<code>backend/src/guard-portal/guard-portal.service.ts</code>
Field & admin UI	<code>frontend/src/app/guard/shifts/[id]/patrols/</code> , <code>frontend/src/app/patrol/</code>

Caveat on the comparison. The vendor columns describe marketed capability, not verified behaviour. Vendor feature lists routinely overstate depth and polish. The gaps identified in this report should therefore be read as *positioning* gaps — things a buyer comparing datasheets would notice — not as proof that either competitor executes those features well.

Our Guard Tour Architecture

- Our tour model is the same fundamental loop both commercial platforms use, expressed as six related entities:
- Checkpoint** — belongs to a Site; carries name, description, location note, an optional QR code value, and an optional GPS geofence (latitude, longitude, radius in metres).
 - PatrolRoute** — a named, ordered tour scoped to a Site.
 - PatrolRouteCheckpoint** — the join table that carries `sequenceOrder` , making the tour an ordered list.
 - PatrolRun** — one execution of a route by one guard on one shift, with `not_started` → `in_progress` → `completed` status and the guard's latest known position.
 - PatrolEvent** — one checkpoint scan, recording timestamp, status, notes, and the server-computed geofence verdict.

- **PatrolEvidence** — photos captured at a scan; multiple photos per checkpoint, stored on local disk with the on-disk filename never exposed to clients.

Three separate portals sit on top of this, matching the shape of both commercial products: an admin monitor, a guard field view, and a client-facing live status endpoint.

Side-by-Side Capability Matrix

Legend: **Yes** = implemented and working · **Partial** = present but incomplete · **No** = not implemented

Core Tour Loop

Capability	Our Platform	Silvertrac	Staffr / TrackTik
Ordered checkpoint tours	Yes — <code>sequenceOrder</code>	Yes	Yes
QR / barcode scanning	No — value stored, never verified	Yes	Yes
NFC tag scanning	No	Yes	Yes
GPS-verified checkpoints	Yes — per-checkpoint geofence, verified server-side	Geofencing at property level	Yes
Virtual / no-scan checkpoints	Yes — effectively the only mode	Yes	Yes — manual check-in
Notes at checkpoint	Yes	Yes	Yes
Photo evidence at checkpoint	Yes — multi-photo per scan	Yes	Yes
Audio evidence	No	Yes	Via report forms
Per-checkpoint configurable actions	No	Basic scan only	Yes — forms, exception questions, alerts
Auto-complete when all hit	Yes	Yes	Yes

Supervision and Alerting

Capability	Our Platform	Silvertrac	Staffr / TrackTik
Live supervisor dashboard	Yes — 15-second poll	Yes — Issue Monitor	Yes
Live guard location	Yes — 30-second device push	Yes	Yes
Live map view	No — per-guard Google Maps link only	Yes	Yes
Missed-checkpoint detection	Retrospective — marked at run completion	Yes — real-time alert	Yes — real-time alert
Late / overdue tour alerting	No — no expected timing field	Yes	Yes
Geofence breach visibility	Yes — daily failure count on dashboard	Yes	Yes
Panic / duress alert	Yes — full acknowledge/resolve lifecycle	Not documented	Lone-worker check-ins
Dispatch tasks mid-shift	No	No	Yes

Field Reliability and Reach

Capability	Our Platform	Silvertrac	Staffr / TrackTik
Offline checkpoint scanning	Yes — queued and replayed	Not emphasised	Yes
Offline tour completion	Yes	Not emphasised	Yes
Offline incident reporting	Yes	Not emphasised	Yes
Offline photo capture	No — upload is online-only	Not documented	Not specified
Duplicate-safe replay	Yes — idempotent by action ID	Not documented	Not documented
Native mobile app	No — responsive web only	Yes	Yes

Back Office and Surrounding Modules

Capability	Our Platform	Silvertrac	Staffr / TrackTik
Client portal	Yes — live site status, incidents, reports	Yes	Yes
Incident reporting + evidence	Yes	Yes	Yes
Scheduling / shifts	Yes	Time clock only	Yes
Timesheets and payroll inputs	Yes	No	Yes
Invoicing and rate cards	Yes	No	Yes
Multi-tenant + RBAC	Yes — tenant-scoped throughout	Per-instance	Yes
Audit logging	Yes	Not documented	Not documented
Parking management	No	Yes	No
Visitor management	No	No	Yes
Asset tracking	No	No	Yes
Post orders / pass-down notes	No	Yes	Yes

Where We Lead

Panic / duress alerting with a managed lifecycle

We have a dedicated `EmergencyAlert` entity with a genuine `ACTIVE → ACKNOWLEDGED → RESOLVED` workflow that records who acknowledged and who resolved each alert. It is deliberately independent of both incidents and patrol runs, so a guard can raise an alert with no active shift or tour. Neither vendor profile describes an equivalent; TrackTik's nearest analogue is scheduled lone-worker check-ins, which is a different and weaker safety mechanism.

Offline-first field operation

Checkpoint scans, tour completion, incident creation, and shift check-in/check-out all queue locally and replay on reconnect. Replay is idempotent — each action carries a client-generated ID, already-synced actions are skipped, and failures are recorded with an error message rather than silently dropped. This puts us clearly ahead of Silvertrac, which does not emphasise offline support at all, and on par with TrackTik.

Server-authoritative geofence verification

The guard's device submits raw coordinates only; it never submits a verdict. The server computes the outcome and writes one of five explicit states: `SUCCESS`, `OUTSIDE_GEOFENCE`, `LOCATION_UNAVAILABLE`, `NO_GEOFENCE_CONFIGURED`, or `INVALID_LOCATION`. Radius is configurable per checkpoint between 5 m and 2,000 m, defaulting to 50 m. This is a stronger anti-tampering posture than a client-reported pass/fail, and the distinction matters in any dispute over whether a guard was genuinely on site.

Privacy-conscious location handling

We deliberately store only the guard's *latest* position on the patrol run, overwritten on each update, plus one point per checkpoint scan. There is no continuous location history table. This is a defensible design choice in jurisdictions with strict worker-surveillance rules, and it is a point neither vendor makes.

Back office already integrated

Shifts, timesheets, invoices, and rate cards are part of the same platform and the same tenant model. Against Silvertrac this is a decisive advantage — Silvertrac is reporting-centric and stops at a time clock. Against TrackTik it is parity on scope, though TrackTik's modules are more mature.

Where We Fall Short

1. Physical scan verification does not exist CRITICAL

This is the single most significant gap. A `qrCodeValue` field exists on the Checkpoint model and is editable in the admin UI, but nothing ever reads it back. The scan endpoint accepts only notes, status, latitude, and longitude — there is no QR value in the request payload, no comparison in the service layer, and no camera scanner, barcode API, or NFC reader anywhere in the frontend. No QR or barcode library appears in the frontend dependencies.

The practical consequence is that **every checkpoint we support is what Silvertrac calls a "virtual" checkpoint**. A guard opens the tour, taps a button in a list, and confirms a modal. Our GPS geofence is a meaningful substitute — it does verify physical presence within a radius — but it is not the same proof as requiring contact with a tag affixed at the location, and GPS accuracy degrades badly indoors, in stairwells, and in basements, which is precisely where many checkpoints live. On a feature matrix, this is the one row where we lose outright to both competitors.

2. No proactive missed or late tour alerting **CRITICAL**

Missed checkpoints are detected only reactively: when a run is completed, any unscanned checkpoint is auto-marked as missed. There is no expected-timing field on the route checkpoint model and no scheduled job anywhere in the patrols module. Both competitors alert a supervisor *while the tour is running late*, which is the point at which intervention is still possible. We surface the same information after the fact, on a dashboard, where it becomes a reporting statistic rather than an operational signal.

3. No per-checkpoint configurable actions **IMPORTANT**

TrackTik's clearest differentiator is that each checkpoint can be configured to open a report form, pose an exception question, display a message to the officer, or fire an alert. Our Checkpoint model carries only descriptive fields — name, description, location note. Every checkpoint behaves identically. Silvertrac does not have this either, so this is a gap against the enterprise tier specifically.

4. No live map **IMPORTANT**

The monitor page lists active runs and links out to Google Maps per guard. Both competitors render all guards on a single map with tour progress overlaid. Functionally this is cosmetic — the underlying location data is already there and already polling — but it is the screen that dominates every product demo, and its absence is immediately visible in a side-by-side evaluation.

5. Offline photo capture is unsupported **IMPORTANT**

Checkpoint scans queue offline, but evidence upload requires connectivity. A guard patrolling a dead zone can record that they visited a checkpoint but cannot attach the photograph that proves what they found there — and by the time they regain signal, the moment has passed.

6. No native mobile application **IMPORTANT**

Our guard experience is responsive web. Both competitors ship native apps with app-store listings and public ratings. Native access matters here beyond preference: NFC reading, reliable background location, and camera integration are all substantially better supported natively, which means this gap partly gates the fixes for gaps 1 and 5.

7. Surrounding modules not in scope **MINOR**

We do not have parking management or a time clock with pass-down notes and post orders (Silvertrac), nor visitor management, asset tracking, or scheduled lone-worker check-ins (TrackTik). Audio evidence is also absent. None of these are core to the tour loop, and each should be treated as a separate product decision rather than a gap to close reflexively.

Recommended Priorities

#	Action	Effort	Rationale
1	QR scan verification	Low	Highest value per unit of effort. Add the QR value to the scan request, compare it server-side, and add a browser-based camera scanner. The verification-status pattern built for geofencing can be extended directly rather than reinvented. This closes our most visible competitive weakness.
2	Expected timing + scheduled alerting	Medium	Add an expected offset to each route checkpoint, run a scheduled job to detect overdue scans and late runs, and route the notification through the existing emergency-alert delivery path. Converts missed-checkpoint data from a report into an operational signal.
3	Live map on monitor page	Low	All required data already exists and already polls. Primarily a demo and evaluation concern, but disproportionately visible for the effort involved.
4	Offline photo queueing	Medium	Completes the offline story that is otherwise one of our stronger differentiators. Requires local blob storage and a deferred upload path.
5	NFC scanning	High	Browser NFC support is limited and inconsistent. Realistically depends on a native app, so it should be sequenced behind that decision rather than attempted independently.
6	Per-checkpoint actions	High	Substantial schema and UI work. Worth doing only if we are targeting enterprise accounts where TrackTik is the incumbent being displaced.
7	Dispatch tasks	High	A new entity and a new guard-facing surface. Defer unless customers ask for it specifically.

Summary Assessment

Our Guard Tour implements the same fundamental loop as both commercial platforms — ordered checkpoints, verified scans, real-time supervisor visibility, photo evidence, and a client-facing view. In several respects the implementation is stronger than what either vendor documents: server-authoritative geofence verification, an idempotent offline queue, a genuine panic-alert lifecycle, tenant-scoped multi-tenancy, and audit logging throughout.

Positioned against the two competitors, we sit **ahead of Silvertrac on scope** — we carry scheduling, timesheets, and invoicing that Silvertrac does not — and **behind TrackTik on tour sophistication**, lacking configurable checkpoint logic and dispatch tasking.

Two gaps, however, cut across that positioning and would be noticed by any evaluator regardless of which competitor they are comparing us to: **the absence of physical scan verification**, and **the absence of proactive alerting while a tour is still running**. Both are addressable with modest effort against the existing architecture, and both are considerably more valuable than any of the surrounding modules we currently lack.

Competitor capability data is drawn from the *Guard Tour Software Research Report* dated September 14, 2026, which was compiled from vendor documentation rather than hands-on evaluation. Our own capability data was derived by direct source-code review on the same date.

and reflects the state of the `fix/cors-aegislead-domain` branch.